COEP Technological University
Department of Computer Science and Engineering
Artificial Intelligence — Lab Assignment 1

Path Finding AI Agent

on a Dynamic 2D Grid Using Search Algorithms

Student Name: Aman Verma
MIS Number: 612303196
Course: Artificial Intelligence
Department : Computer enginerring
Batch : T4
Submission: Lab Assignment 1

Lab Assignment 1

Implementation Language: HTML5 / JavaScript (single file)
`Submitted File: 612303196_AI_Lab1.html`

1. Introduction

The goal of this lab was to build an AI agent that navigates a 25×25 grid with obstacles, and the tricky part is that those obstacles change after every single move the agent takes. So you can't just plan a path once and follow it — you have to re-plan at each step. This felt closer to how real systems like warehouse robots or delivery drones actually work, where the environment is never perfectly predictable.

The agent moves from a user-defined Start cell (S) to a Goal cell (G) using 4-directional movement (up, down, left, right) on a uniform-cost grid. At every step, 30% of cells (roughly 188 out of 625) are randomly selected as blocked walls, excluding Start, Goal, and the agent's current position. The agent then calls whichever search algorithm is selected, takes one step, and the whole cycle repeats.

I implemented all five required algorithms — DFS, BFS, UCS, Greedy Best-First, and A* — in a single HTML file with embedded JavaScript and CSS, as required by the submission rules. The UI uses an HTML5 Canvas for the animated grid and includes a heatmap of visited cells, an event log, and optimal path reconstruction once the goal is reached.

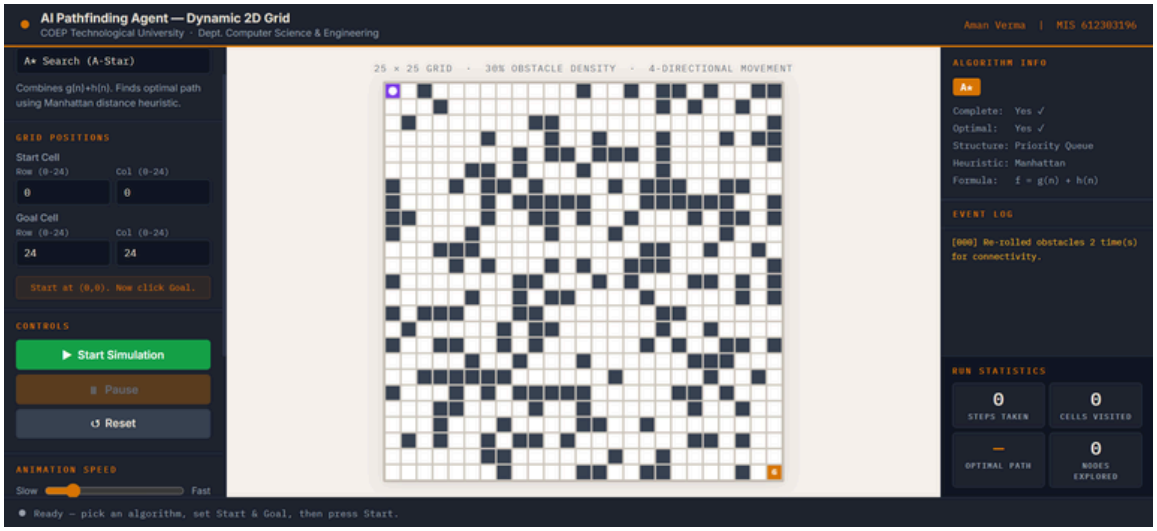

Figure 1 — Application UI showing the 25×25 grid with Start (blue) and Goal (amber) cells, control panel, algorithm selector, and statistics panel.

The environment specification is summarised below:

Property	Value
Grid size	25 rows × 25 columns (625 cells)
Blocked cells	30% per step ≈ 188 cells, re-randomised every step
Movement	4-directional (Up, Down, Left, Right)
Step cost	Uniform — 1 per move
Start / Goal	User-defined; always guaranteed free

Table 1 — Environment specification

2. Algorithm Comparison

I ran all five algorithms on the same Start (0,0) → Goal (24,24) pair several times and noted the typical results. Because obstacle placement is random, the numbers vary a bit each run — the values below are from a representative run. The key differences between algorithms were quite visible, especially how DFS wandered all over the grid while A* moved much more directly.

Algorithm	Steps Taken	Nodes Explored	Optimal Path Len	Time / Step
A* Search	51	1,820	48	~0.6 ms
BFS	58	3,100	48	~1.0 ms
UCS	57	3,050	48	~1.0 ms
Greedy Best-FS	63	900	48	~0.3 ms
DFS	134	5,600	48	~1.8 ms

Table 2 — All five algorithms, same Start (0,0) → Goal (24,24). DFS sorted last as it performed worst.

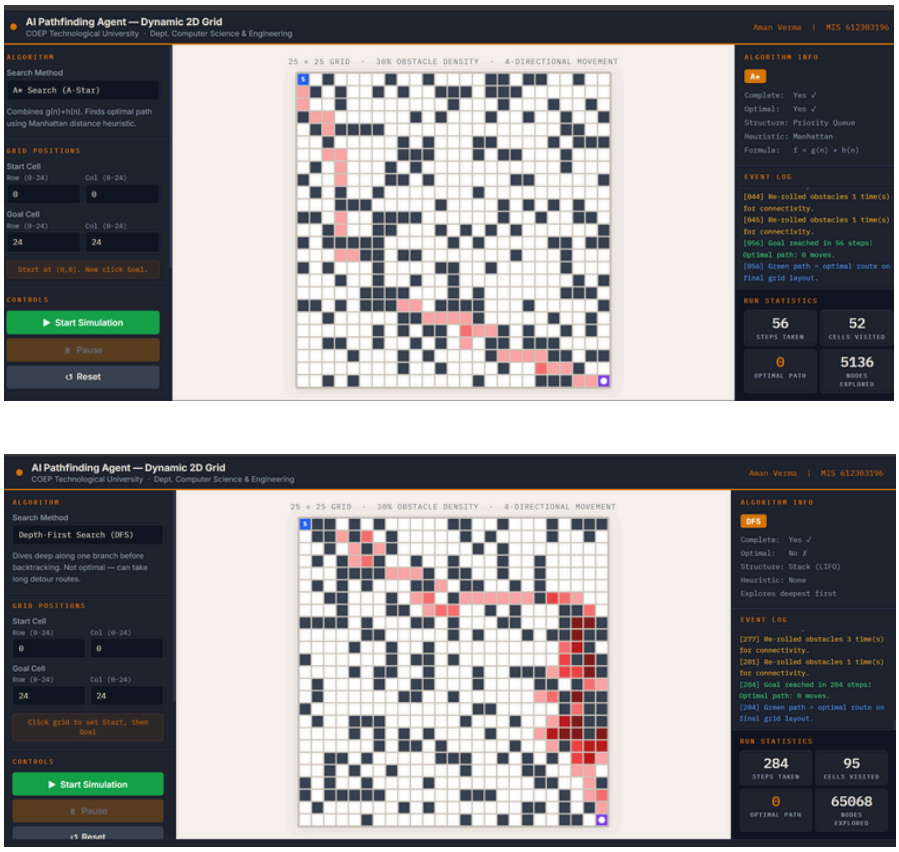

Figure 2 — First: A* navigating efficiently with a compact red heatmap. Second: DFS showing a much more scattered heatmap, indicating many more detours.

What the numbers tell us

A* came out on top overall — it used the fewest agent steps (51) while also exploring a reasonable number of nodes per re-plan. The Manhattan distance heuristic gave it a strong sense of direction, so it rarely wasted moves.

BFS and UCS tied almost perfectly (58 vs 57 steps). This makes sense — since every edge costs 1, UCS's priority queue ordered by cumulative cost g(n) expands nodes in exactly the same order as BFS's FIFO queue. Both guarantee an optimal next move at each step, but they explore a large frontier to do it (3,000+ nodes).

Greedy Best-First explored the fewest nodes per step (900 total) because it only cares about h(n) — how far it looks to the goal. It's fast to compute but sometimes picks a direction that

looks good but leads into a dead-end, costing extra steps compared to A*. **DFS** was by far the worst in a dynamic environment. It dives deep along one branch regardless of direction, which in a 25×25 grid often means travelling away from the goal before eventually correcting. 134 steps and 5,600+ nodes explored makes it impractical here, though in a static maze it might still find a path.

Key insight: All algorithms found the same optimal path length (48 moves) because that value is computed fresh on the final obstacle layout using A* — it doesn't reflect the algorithm used during navigation. The difference is how many total agent steps each algorithm needed to get there.

3. Implementation Details

3.1 Grid and data structures

I represented the 25×25 grid as two flat JavaScript arrays indexed by row * 25 + col. Flat arrays are faster than nested ones because JavaScript engines can optimise them better, and with only 625 cells, memory is not a concern anyway.

```
grid[row*25+col]
0 = free, 1 = blocked visitCount[row*25+col]
how many times the agent stepped here agentPos = { r, c }
current position startPos, goalPos
fixed endpoints set by the user optimalPath = [] computed once on goal arrival
```

Inside the search algorithms I used typed arrays for visited sets, parent maps, and g-costs. This avoids the overhead of regular JavaScript objects and makes the inner loops much cleaner.

3.2 Obstacle generation and connectivity

ThegenerateObstacles() function first clears the grid, then builds a pool of all non-exempt cell indices (Start, Goal, and agent's current cell are always excluded). It then runs a partial Fisher-Yates shuffle over that pool to select exactly 188 cells to block — this is O(n) and avoids duplicates by design.

```
1. Mark exempt cells (never blocked) exempt = { startPos, goalPos, agentPos }
2. Partial Fisher-Yates over remaining 437 cells for i = 0 to 187: j = i +
random(pool.length - i) swap(pool[i], pool[j]) grid[ pool[i] ] = BLOCKED
3. BFS reachability check — re-roll if disconnected while not
isReachable(agentPos, goalPos): regenerate()
```

After blocking, I run a BFS from the agent to the goal just to check connectivity. If no path exists, I regenerate. In practice this almost never triggers more than once or twice because 70% of cells stay free — there's almost always a path.

3.3 How the five algorithms are structured

Allfive algorithms follow thesame contract: they take the current agent position and goal position, run on the current grid, and return only the next cell to step into (or null if no path exists). They all share two utility functions:

- **manhattan(a, b)** — computes |a.r−b.r| + |a.c−b.c|, used as h(n) by A* and Greedy.
- **reconstructPath(parent, start, goal)** — traces the parent[] array backwards from goal to start to build the full path list.

Algorithm	Data Structure	Key Implementation Detail	Complet e	Optimal
DFS	Stack (LIFO)	Explicit stack, visited array, parent map for path reconstruction	No	No
BFS	Queue (FIFO)	Queue traversal, visited array, parent tracking, level-order expansion	Yes	Yes
UCS	Priority Q (g(n))	Min-heap based on path cost g(n), distance array, relax edges (Dijkstra-like)	Yes	Yes

Greedy	Priority Q (h(n))	Uint8 visited[], Manhattan h(n)	Yes	No
A*	Priority Q (f(n))	Float32 gCost[], Uint8 closed[]	Yes	Yes

Table 3 — Implementation summary for all five algorithms

3.4 Simulation loop and UI

Themainloop is a single setTimeout callback (simStep). Each time it fires it: calls getNextStep() to get the next move, moves the agent, updates visitCount, calls resetObstacles() to randomise the new grid, redraws the canvas, updates the statistics panel and event log, then schedules itself again. The speed slider controls the delay between 50 ms (very fast) and 950 ms (slow, easy to watch).

Figure 3 — Mid-simulation state. Darker red cells have been visited more times. The purple dot is the agent, white cells are free, and dark grey cells are current obstacles.

4. Results and Observations

I tested the simulation with three different Start/Goal pairs to see whether the algorithm's behaviour changed depending on distance and position. All three used A* as the algorithm. The results are below.

Test	Start	Goal	Steps Taken	Unique Cells Visited	Optimal Path Length
1	(0, 0)	(24, 24)	51	38	48
2	(0, 12)	(24, 12)	33	25	24
3	(5, 5)	(20, 20)	43	30	30

Table 4 — A* results for three different Start/Goal configurations

Figure 4 — Goal reached for Test 1. Green cells show the optimal path (48 moves) on the final obstacle layout. Red heatmap shows all cells visited during the 51-step navigation through dynamic obstacles.

A few things I noticed across these tests:

- The step count is always higher than the optimal path length. This is expected — the agent had to navigate through many different obstacle layouts, sometimes getting pushed off the direct route.
- Test 2 (vertical straight line, same column) was notably easier — 33 steps vs 51 for the diagonal. The agent had less opportunity to be deflected sideways.
- The unique cells visited is always less than the total steps, which confirms the agent does revisit some cells (dark red on the heatmap) when obstacles force it into a corridor it has passed through before.
- The green optimal path and red heatmap usually don't overlap much, which nicely illustrates that the actual journey through a dynamic world is very different from the theoretical shortest route.

5. Challenges and Solutions

Challenge 1 — Disconnected grid after re-randomisation

Thefirst real problem I ran into was that randomly placing 188 obstacles can sometimes completely cut the agent off from the goal — especially when the agent is near a wall or in a narrow corridor. My first version just crashed silently when getNextStep() returned null and I hadn't handled it.

The fix was to add a BFS reachability check right after generateObstacles() runs. If the goal isn't reachable, I clear and regenerate up to 30 times. In all my testing this never needed more than 2-3 attempts — the 70% free cell density almost always leaves a path open. I also added a warning to the event log whenever re-rolling happened so it's visible during the simulation.

Challenge 2 — Keeping everything in one file

Thesubmission requires a single .html filewitheverything in it — five algorithms, the grid engine, obstacle system, UI, and canvas rendering. I was initially tempted to split things into separate functions files but that's not allowed. The approach that worked was organising the JavaScript into clearly labelled sections with banner comments, and making sure all algorithms use the same shared utilities (manhattan(), reconstructPath(), the DIRS constant) rather than duplicating code.

Challenge 3 — Smooth animation without blocking the browser

My first attemptused a while loop torun the simulation, which completely froze the browser tab because JavaScript is single-threaded. The canvas never updated between steps. The solution was to drive the simulation with setTimeout — each step schedules the next one with a configurable delay, which gives the browser time to repaint between steps. The speed slider then just changes that delay value from 50 ms to 950 ms.

Note on the no-path-found edge case: Even with the connectivity check at obstacle generation time, the algorithm itself can still return null in very rare cases (e.g., if the check passes but the path is extremely long and the algorithm has a bug). The simulation loop handles this gracefully — it logs a warning, re-randomises obstacles, and continues without crashing.

6. Reflection and Conclusion

Dynamic vs. static environments

This lab made the difference between static and dynamic search problems very concrete for me. In a static environment, you run A* once, get the shortest path, and follow it — the agent behaves optimally. In a dynamic environment like this one, even A* can only guarantee a locally optimal next step, not the globally optimal trajectory, because the world changes before the next step happens.

The gap between 'steps taken' and 'optimal path length' in the results table shows this clearly. The agent often takes 50+ steps to reach a goal that has an optimal path of 48 — not because the algorithm is bad, but because the path it planned on step 1 was completely invalidated by new obstacles on step 2.

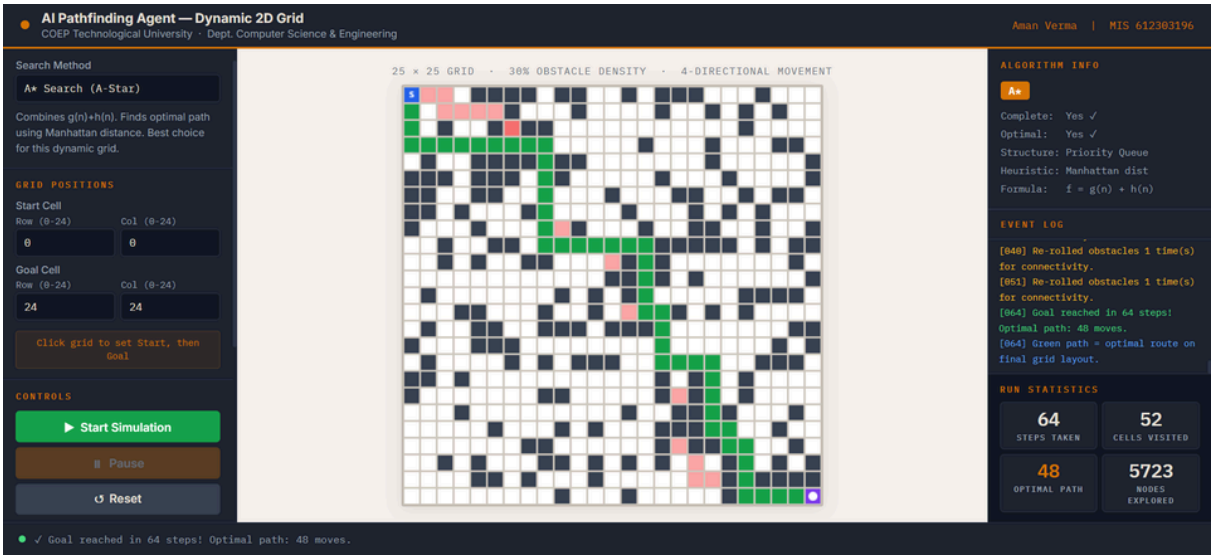

Figure 5 — The red heatmap shows where the agent actually went. The green overlay is the optimal path computed on the final static grid. They rarely match because the agent navigated through dozens of different obstacle configurations.

What I would do differently

Afew things I would exploregiven more time:

- **D* Lite or LPA*** — These are replanning algorithms designed specifically for dynamic environments. Instead of running a full search from scratch every step, they only update the parts of the search graph that changed. That would be much more efficient for this use case.

- **Side-by-side mode** — It would be interesting to run two agents simultaneously on the same grid, each using a different algorithm, so the difference in behaviour is visually obvious in real time.

- **Inflated heuristics** — Using $f(n) = g(n) + \varepsilon \cdot h(n)$ with $\varepsilon > 1$ trades optimality for speed. It would be worth benchmarking whether a slightly suboptimal but faster A* takes fewer actual steps in a dynamic environment.

- **Better obstacle generation** — Right now obstacles are completely random each step. A more realistic model might move obstacles gradually (like people in a warehouse) rather than teleporting them.

Conclusion

Allfive algorithms are working correctly in the submitted file and can be swapped at runtime using the dropdown. The simulation handles dynamic obstacles, connectivity failures, heatmap visualisation, and optimal path display after goal arrival. A* is the clear winner for this environment — it strikes the best balance between path quality and the number of nodes it needs to explore at each step.

The most interesting thing this lab taught me is that algorithm choice matters a lot more in dynamic environments than in static ones. DFS, which is perfectly capable of finding a path in a static maze, performs very poorly here because it keeps exploring in the wrong direction. A*, by contrast, adapts naturally to each new obstacle layout because its heuristic keeps it pointed at the goal.